

tempssm 詳細マニュアル

Version 0.3.0

平尾章・馬場真哉・市野川桃子

2026-07-23

概要

`tempssm` は、気温・水温などの温度時系列に対する状態空間解析を行うための R パッケージです。計算バックエンドとして、KFAS R パッケージ (Helske, 2017) を用い、カルマンフィルタリングおよび平滑化によって推定される線形ガウス状態空間モデルを適用します。

主な特徴

- ・ 温度時系列に線形ガウス状態空間モデルを適用
- ・ 温度の動態を潜在状態 (latent state) の次の各成分に分解して表現：長期トレンド、季節変動、自己回帰構造、および任意の外生効果
- ・ 任意の季節周期に対応（ただし実例と検証では、主として月別の温度データを対象）
- ・ 自己回帰成分の次数の任意指定が可能
- ・ 要約、モデル診断、プロットのための S3 メソッドを提供
- ・ モデル評価のための時系列交差検証ツールを提供

既存研究と本パッケージの範囲

`tempssm` は、線形ガウス状態空間モデルを用いて温度時系列を解析するための、対象分野に特化したワークフローを提供します。温度データへの応用に合わせた単一の R パッケージインターフェースの中に、モデル構築、成分抽出、不確実性の要約、残差診断、可視化、時系列交差検証を統合しています。

本パッケージは、線形ガウス状態空間モデリング、カルマンフィルタリング、カルマン平滑化を含む、確立された統計的方法論に基づいています。モデル推定は、R における状態空間モデルの汎用的な枠組みを提供する KFAS パッケージを通じて行われます。

初期実装は、線形ガウス状態空間モデルを用いて海水温トレンドを解析した Baba et al. (2024) に付随する補足コードに基づいています。この補足コードは以下で公開されています。

<https://github.com/logics-of-blue/sea-temperature-trend-jogashima>

この先行実装と比べて、`tempssm` は、入力検証、文書化された S3 メソッド、テスト、診断、交差検証ユーティリティ、およびより広範な温度時系列解析のための例を備えた、再利用可能な R パッケージインターフェースへとワークフローを拡張しています。

次節では、`tempssm` で用いる状態空間モデルを説明し、長期トレンド、季節成分、自己回帰成分、外生成分がどのように表現されるかを整理します。

tempssm における状態空間モデル

本パッケージでは、長期トレンド、季節変動、自己回帰成分を明示的に含む温度時系列を記述するために、基本構造時系列モデル（Basic Structural Time Series Model; BSTSM）を拡張したモデルを考えます。観測方程式は次のように表されます。

$$y_t = \alpha_t + v_t. \quad (1)$$

ここで、 t は時点、 y_t は観測された温度、 α_t は潜在状態、 v_t は 観測誤差を表します。

潜在状態は次のように分解されます。

$$\alpha_t = \mu_t + s_t + r_t + E_t. \quad (2)$$

ここで、 μ_t は長期トレンド成分、 s_t は季節成分、 r_t は定常的な 自己回帰成分、 E_t は外生変数の寄与を表します。

レベル成分は、次の 2 次の確率過程に従います。

$$\mu_t = 2\mu_{t-1} - \mu_{t-2} + \zeta_t, \quad \zeta_t \sim \mathcal{N}(0, \sigma_\zeta^2). \quad (3)$$

ここで、 ζ_t は長期トレンド成分の過程誤差です。

季節成分は、総和ゼロ制約をもつモデルとして表されます。

$$s_t = - \sum_{i=t-f}^{t-1} s_i + \omega_t, \quad \omega_t \sim \mathcal{N}(0, \sigma_\omega^2). \quad (4)$$

ここで、 ω_t は季節成分の過程誤差、 f は季節周期を表します（例えば 月別データでは $f = 12$ ）。この定式化により季節効果が識別可能となり、異なる時間分解能をもつ時系列に対応したモデルを構築できます。1 つの完全な季節周期にわたって総和ゼロ制約を課すことで、季節成分は長期的なドリフトを導入することなく、基礎となる長期トレンドからの反復的な偏差を捉えます。季節成分を含まないモデルでは、季節項 s_t はゼロとし、状態方程式から省略されます。

自己回帰成分は、 l 次の自己回帰（AR）過程に従います。

$$r_t = \phi_1 r_{t-1} + \phi_2 r_{t-2} + \cdots + \phi_l r_{t-l} + \tau_t, \quad \tau_t \sim \mathcal{N}(0, \sigma_\tau^2). \quad (5)$$

ここで、 τ_t は自己回帰成分の過程誤差を表します。過程誤差項（ ζ_t 、 ω_t 、および τ_t ）は互いに独立で、正規分布に従うと 仮定します。

外生成分は次のように定義されます。

$$E_t = \beta_1 x_{1,t} + \beta_2 x_{2,t} + \cdots + \beta_m x_{m,t}. \quad (6)$$

外生項 E_t は外部要因の影響を表します。外部要因には、大規模な気候指数、地域的な環境変数、あるいは観測された温度時系列に関連する物理的根拠をもつ予測変数などが含まれます。各外生変数は、時間に依存しない回帰係数（ $\beta_1, \beta_2, \dots, \beta_m$ ）を通じて線形にモデルへ組み込まれ、その寄与の大きさと方向を潜在状態成分と同時に推定できます。

観測データにモデルを適用する際、推定されるパラメータの総数（ k ）は、自己回帰成分の次数と、モデルに含まれる外生変数の数に依存します。例えば、2 次の自己回帰成分をもち、外生共変量を含まないモデルでは、観測誤差分散、長期トレンド・季節・自己回帰成分に対応する過程誤差分散、および 1 次・2 次の 自己回帰係数（ ϕ_1 と ϕ_2 ）の計 6 個のパラメータが推定されます。一般に、 l 次の自己回帰成分と m 個の外生変数をもつ場合、パラメータの総数は $k = 4 + l + m$ です。

パラメータ推定手順の中核的な実装は、Baba et al. (2024) によって提供された 補足コードに基づいています。パラメータ推定は、Helske (2017) が推奨する 2 段階の 最適化戦略によって行われます。第 1 段階では、ユーザー指定の初期値を用いて Nelder-Mead 法 (Nelder & Mead, 1965) によりモデルパラメータを推定します。得られた推定値を、第 2 段階の BFGS アルゴリズム (Shanno, 1970) による最適化の初期値として用います。本パッケージの主要関数である `tempssm()` は、モデルの構築からパラメータ推定までを一挙に実施し、フィッティング推定値と平滑化推定値の両方を `tempssm` オブジェクトに返り値とし格納します。一方で、S3 メソッドを介して示される要約、モデル診断、プロットで示す結果はすべて平滑化推定値に基づいています。

使用方法

環境設定

以下の例を実行するため、以下の R パッケージを読み込みます。未インストールのパッケージがあれば、適宜、インストールください。

```
## Set libraries
library(tempssm)
library(forecast)

library(purrr)
library(tibble)
library(readr)
library(dplyr)
library(ggplot2)

library(patchwork)
```

入力データ形式

`tempssm` への入力データは、等間隔の時系列を表す R の `ts` オブジェクトとして与える必要があります (`?stats::ts` または <https://stat.ethz.ch/R-manual/R-devel/library/stats/html/ts.html> を参照してください)。

データ準備を支援するため、本パッケージには、外部の観測データを `ts` オブジェクトへ変換するユーティリティ関数が含まれています (付録を参照)。

演習 I: 単変量の温度時系列への状態空間モデルの適用

目的

演習 I では、単変量の温度時系列に線形ガウス状態空間モデルを適用する方法を実践します。外生変数を含めない基本的なモデリングの導入に位置づけられます。あわせて自己回帰的な動態が果たす役割を検討します。

海面水温 (SST) データセットの読み込み

本パッケージには、以下の海面水温 (SST) のサンプルデータセットが含まれています。

- データセット: 山口県沖の月別海面水温 (SST)
- 単位: °C
- 期間: 2002 年 2 月から 2023 年 12 月

このデータセットは、気象庁ウェブサイト (<https://www.jma.go.jp/jma/indexe.html>) から取得した日別 SST データセットを月別に集約したものです。

```
data(yamaguchi_sst) # load a ts object of SST off Niigata
head(yamaguchi_sst)
```

```
##           Jan      Feb      Mar      Apr      May      Jun
## 1982 14.85516 13.36500 13.55645 14.29933 17.72419 21.53000
```

```
summary(yamaguchi_sst)
```

```
##           Temp
## Min.      :11.45
## 1st Qu.:15.13
```

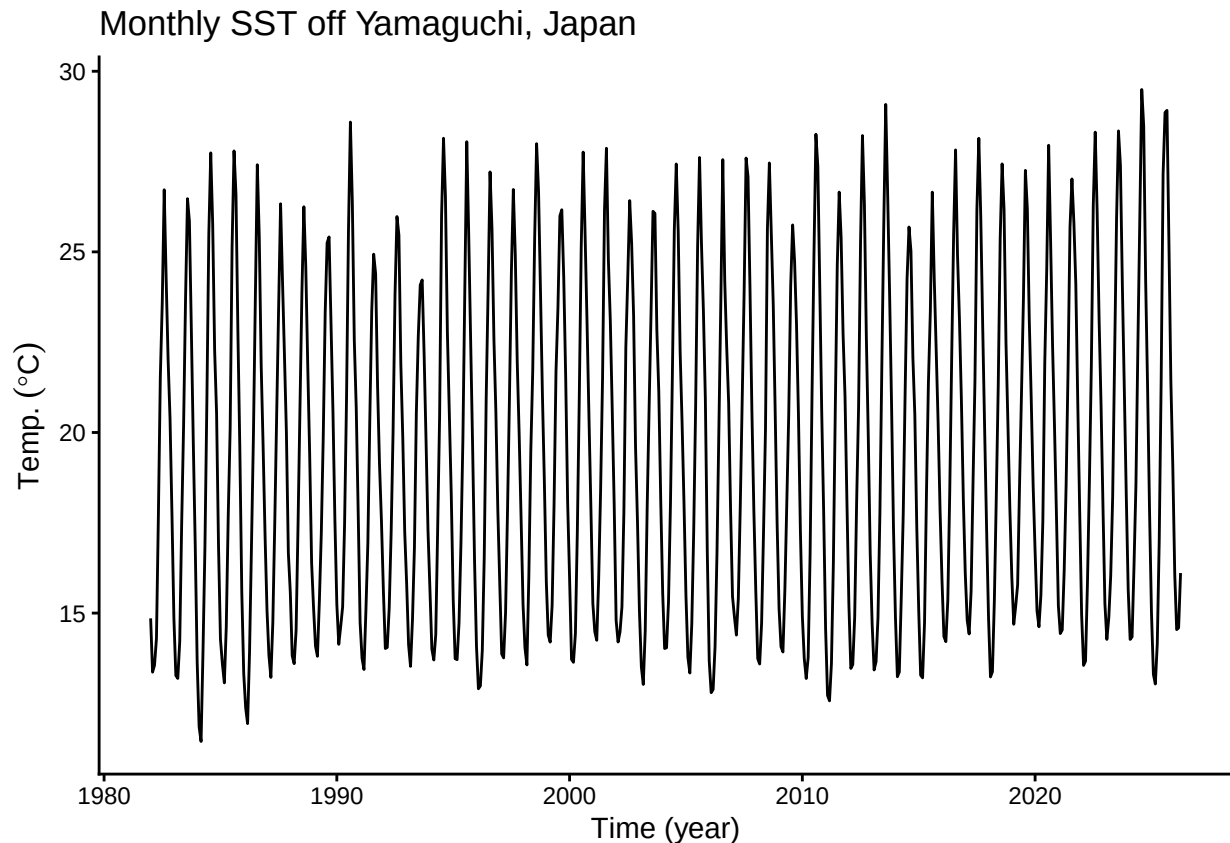
```
## Median :18.84
## Mean   :19.54
## 3rd Qu.:23.63
## Max.   :29.49
```

月別 SST 時系列のプロット

まず、月別 SST 時系列を可視化し、見かけのトレンド、季節変動、欠測値の有無を含む全体的な構造を確認しておきます。

```
plt_yamaguchi_sst <- forecast::autoplot(yamaguchi_sst) +
  labs(y = expression(Temp.~(degree*C)),
       x = "Time (year)") +
  ggtitle("Monthly SST off Yamaguchi, Japan") +
  theme_classic()

plot(plt_yamaguchi_sst)
```



SST の全体平均は約 19.5 °C であり、明瞭な季節変動パターンが認められます。この時系列には欠損値は含まれていません。観測期間を通じて SST は上昇しているように見えますが、年変動も認められるため、生の時系列だけから長期トレンドを分離して把握することは容易ではありません。

線形ガウス状態空間モデルの適用

本パッケージのコア関数 `tempssm()` に温度時系列データの `ts` オブジェクト（ここでは `yamaguchi_sst`）を与えて、実行すると、モデル構築からパラメータ推定までが一挙に処理されます。返り値として、フィルタリング推定値、平滑化推定値などの結果に加えて、構築されたモデルや入力データなどが S3 クラスの `tempssm` オブジェクト（ここでは `res_ar1`）に格納されます。デフォルトでは `tempssm()` は 1 次の自己相関回帰モデ

ルをあてはめます。

```
# model with first-order autoregressive component
res_ar1 <- tempssm(yamaguchi_sst) # (ar_order=1: default)
summary(res_ar1)
```

```
## tempssm summary
## -----
## Call:
## tempssm(temp_data = yamaguchi_sst)
##
## Model fit:
##   Likelihood type: marginal
##   Log-likelihood : -437.2
##   k                : 5
##   Diffuse states : 13
##   Converged       : TRUE
##
## Variance parameters:
##   Observation (H): 4.542027e-14
##   State (Q trend): 9.936473e-08
##   State (Q season): 8.408352e-56
##   State (Q ar): 0.3145626
##
## Components of auto-regression:
##   Order of AR: 1
##   Coefficient of AR1: 0.6202321
```

結果の要約から、モデルが収束したこと (Converged: TRUE) を確かめます。その他に統計量として、パラメーター数 (k)、対数尤度 (Log-likelihood)、尤度タイプ、散漫初期化された状態数が示されています。パラメーター推定値の内訳は次のとおりです：観測誤差の分散 (H)、長期トレンド成分の過程誤差の分散 (Q trend)、季節周期成分の過程誤差の分散 (Q season)、自己回帰成分の過程誤差の分散 (Q trend)、1 次の自己回帰係数 (AR1)

対数尤度と対応する推定パラメーター数は、`logLik()` を用いて推定済みの `tempssm` オブジェクトから直接取得できます。

```
ll <- logLik(res_ar1)
ll

## 'log Lik.' -437.1989 (df=5)

attr(,"df") # number of parameters
```

```
## [1] 5
```

`tempssm()` は、デフォルトでは KFAS の周辺尤度 (marginal likelihood) を用いてパラメーター推定をおこないます。一方で、`marginal = FALSE` を明示すれば、散漫尤度 (diffuse likelihood) も利用できます。選択された尤度タイプは推定済みの `tempssm` オブジェクトに保存され、`logLik()` や `summary()` などの下流メソッドでも一貫して用いられます。

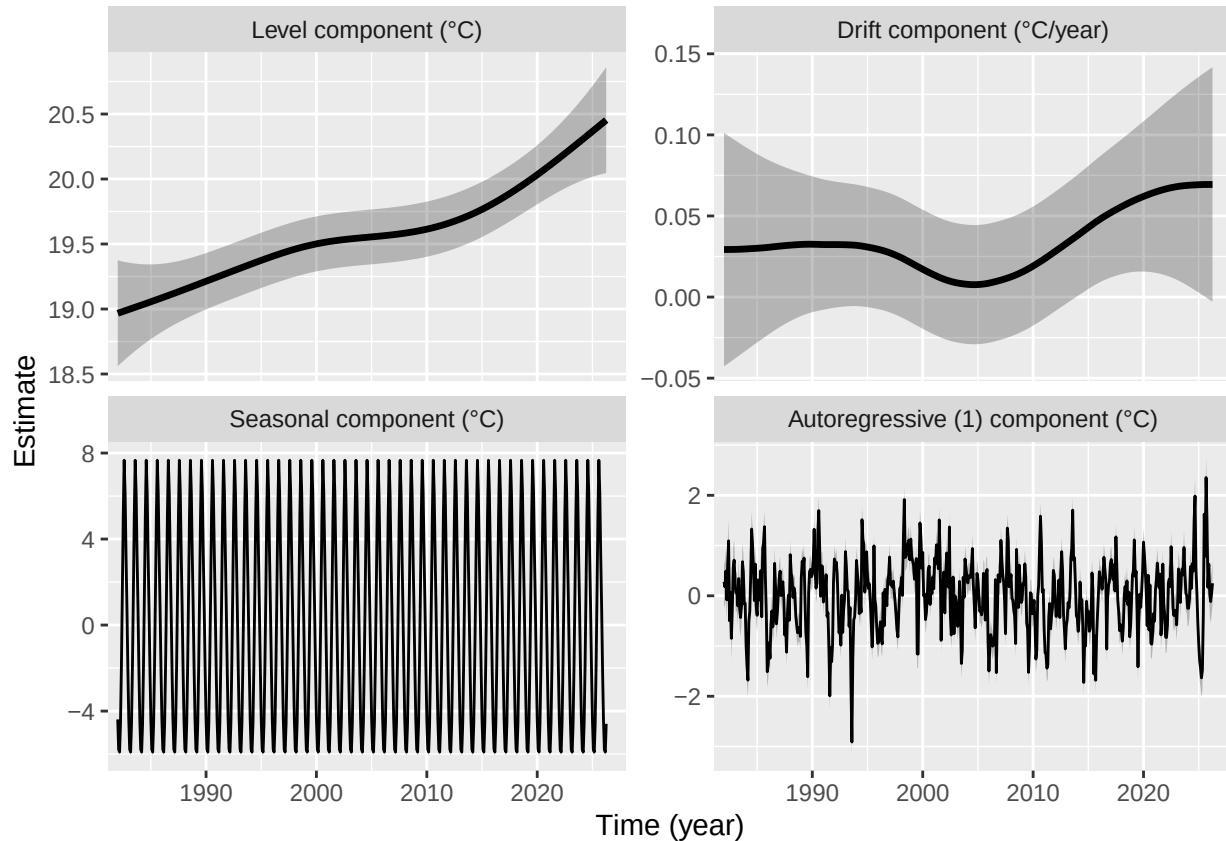
本パッケージでは、`tempssm` オブジェクトに対して AIC を計算しない方針を採用しています。一方で、対数尤度やパラメーター数は `logLik()` を通じて取得できます。これらをどのようにモデル評価に用いるかは、ユーザー自身のモデル比較の前提に基づいて判断してください。

長期トレンド、ドリフト、季節成分、自己回帰成分のプロット

状態空間モデルから対応する潜在成分を抽出し、推定された温度レベルの長期的な変化と、その変化率（ドリフト）を可視化します。あわせて季節変動と自己回帰的依存性もプロットすることで、基礎となるトレンド構

造をより明確に確認できます。

```
# plot all components at once  
plot(res_ar1)
```



パネル左上の長期トレンドは、解析期間を通じて SST 上昇していくパターンを示しています。このグラフの例において、解析期間を通じた SST の平均的な年上昇率は約 0.036 °C です。

しかしながら、SST の変化率そのものも時系列に沿って変化しているようです。パネル右上の変化率（ドリフト）に注目すると、2000 年代に 変化率が 0（または 0 の近傍）に低下した後、2010 年以降に上昇したことが読み取れます。灰色の網掛け部分は、推定された潜在状態の 95% 信頼区間を表し、各推定成分に伴う不確実性を示しています。

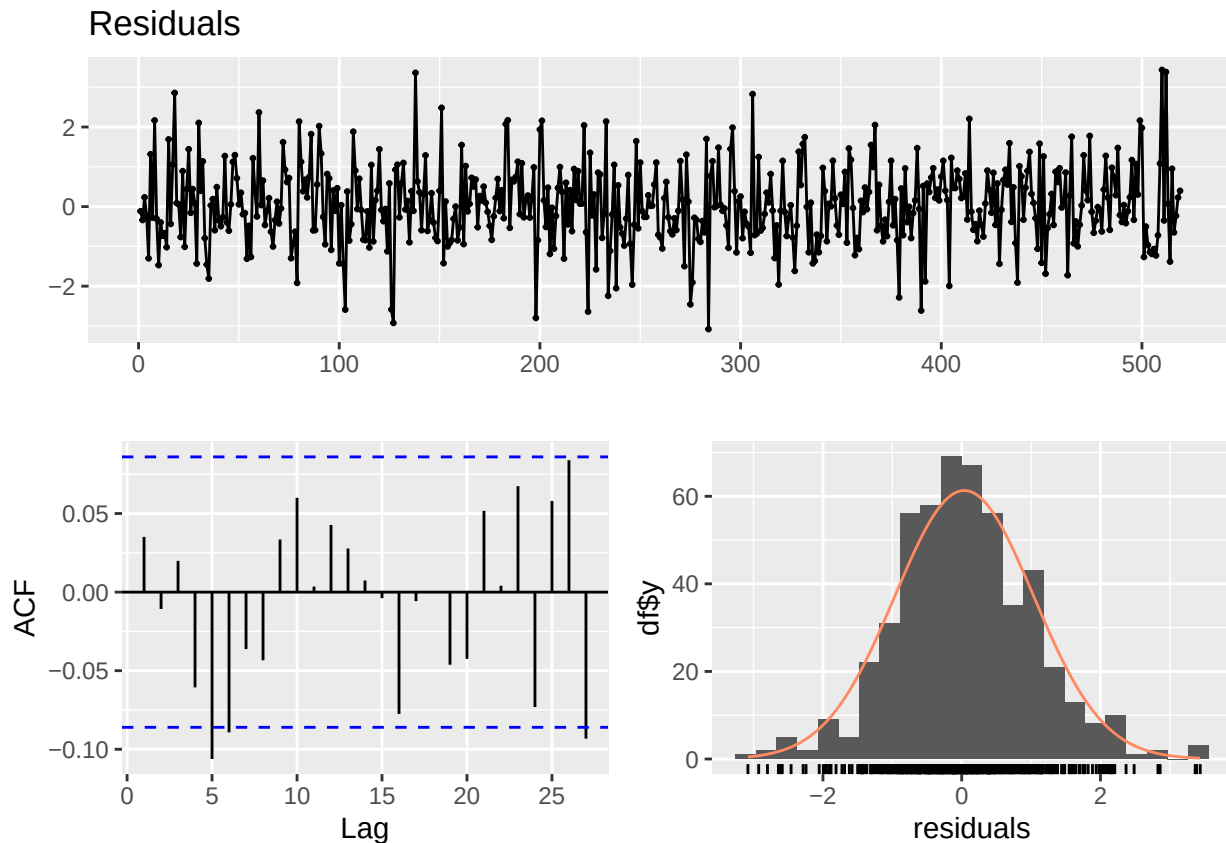
標準的なプロット用インターフェースは `plot(res)` です。ggplot2 形式のインターフェースである `autoplot(res)` も利用でき、デフォルトでは同じ成分プロットを生成します。個別の成分について ggplot オブジェクトを取得するには、以下のように `autoplot()` の `component` 引数を指定します。

```
# plot each of components at once  
plt_level <- autoplot(res_ar1, component = c("level"))  
plt_drift <- autoplot(res_ar1, component = c("drift"))  
plt_season <- autoplot(res_ar1, component = c("season"))  
plt_ar1 <- autoplot(res_ar1, component = c("ar1"))
```

モデル診断

本パッケージでは、推定モデルの残差に注目して、モデルが十分に時系列構造を捉えているかを確認するための診断用関数を提供しています。具体的には、残差の時系列プロット、自己相関、分布、および Ljung-Box 検定を用いて、残存する時間依存性や正規誤差仮定からの大きな逸脱がないかを確認できます。

```
plot_tempssm_residual_diagnostics(res_ar1)
```



モデル診断プロットにおいて、上パネルは残差の時系列プロット、下左パネルは残差の自己相関プロット（ACFプロット）、下右パネルは頻度分布プロットを表します。残差の各パターンについて留意すべき点がないかをチェックしておきます。

この例では、残差 ACF プロットにおいて lag 5 と lag 27 付近に単発の突出が認められます。しかし、連続する複数のラグにわたる有意な自己相関や、季節周期に対応する反復的なパターンは明瞭ではありません。多数のラグを確認する場合には、このような単発の突出が偶然に生じることもあるため、ACF プロットだけでモデル変更を判断するのではなく、Ljung-Box 検定や残差時系列プロットとあわせて解釈します。

```
diag <- diagnose_residuals(res_ar1)
print(diag)
```

```
## # A tibble: 1 x 4
##   lb_stat lb_lag lb_pvalue kurtosis
##   <dbl> <dbl>   <dbl>   <dbl>
## 1    18.1    12     0.111    3.70
```

`diagnose_residuals()` の返り値のうち、`lb_stat`、`lb_lag`、`lb_pvalue` はそれぞれ Ljung-Box 検定統計量、検定に用いたラグ、対応する P 値を表します。月別時系列では、デフォルトで季節周期に対応する lag 12 が用いられます。この例では、lag 12 までの残差自己相関に有意性は認められませんでした。

より長いラグにおける残差自己相関が気になる場合には、検定に用いるラグを手動で指定できます。例えば、次のコードでは lag 24 および lag 36 までの Ljung-Box 検定を追加で実行します。

```
diag_lag24 <- diagnose_residuals(res_ar1, lb_lag = 24)
diag_lag36 <- diagnose_residuals(res_ar1, lb_lag = 36)
```

```
diag_lags <- rbind(diag, diag_lag24, diag_lag36)
diag_lags$check <- c("lag 12", "lag 24", "lag 36")
diag_lags[, c("check", "lb_stat", "lb_lag", "lb_pvalue", "kurtosis")]
```

```
## # A tibble: 3 x 5
##   check lb_stat lb_lag lb_pvalue kurtosis
##   <chr>   <dbl> <dbl>   <dbl>   <dbl>
## 1 lag 12    18.1    12     0.111     3.70
## 2 lag 24    30.8    24     0.158     3.70
## 3 lag 36    48.9    36     0.0746    3.70
```

これらの検定結果は、残差 ACF プロットとあわせて解釈します。多数のラグを確認する場合には、少数のラグが偶然に信頼限界を超えることもあります。一方で、周期的または連続的な突出がみられる場合には、時間構造が残っている可能性があります。

自己回帰（AR）次数の検討

本マニュアルでは、tempssm() のデフォルトである AR(1) を作業モデルとして用います。自己回帰成分は、潜在的な水準、ドリフト、季節成分では表現されない短期的な系列依存性を吸収するために組み込まれています。そのため、高い AR 次数を自動的な改善として扱うのではなく、残差診断により自己相関が残っていると考えられる場合に、AR 次数を増減させて検討する方針が適切です。

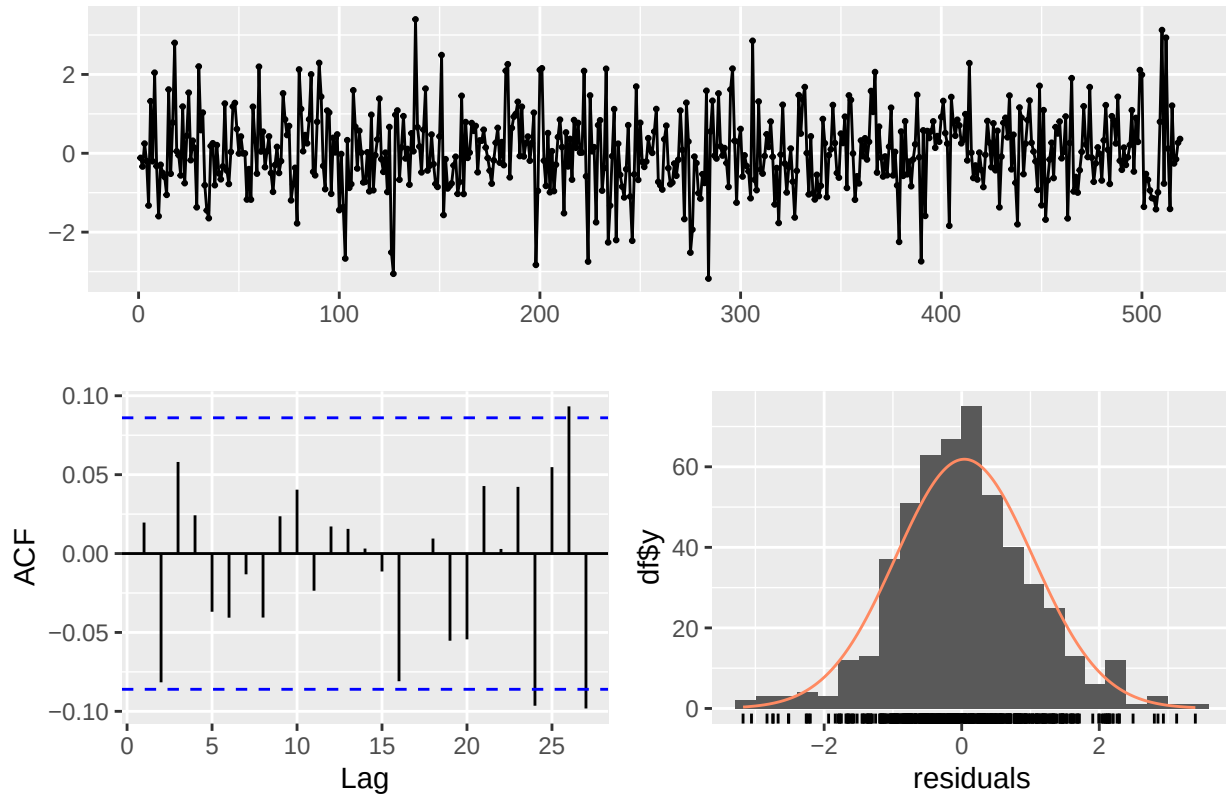
例えば、AR(1) モデルを当てはめた後に残差自己相関が残る場合には、AR(2) モデルを当てはめ、同じ残差診断を繰り返すことができます。

```
# Optional sensitivity check with a second-order autoregressive component
res_ar2 <- tempssm(yamaguchi_sst, ar_order = 2)
summary(res_ar2)
```

```
## tempssm summary
## -----
## Call:
## tempssm(temp_data = yamaguchi_sst, ar_order = 2)
##
## Model fit:
##   Likelihood type: marginal
##   Log-likelihood : -434.26
##   k                : 6
##   Diffuse states : 13
##   Converged      : TRUE
##
## Variance parameters:
##   Observation (H): 0.120908
##   State (Q trend): 2.206262e-07
##   State (Q season): 5.525557e-16
##   State (Q ar): 0.1044204
##
## Components of auto-regression:
##   Order of AR: 2
##   Coefficient of AR1: 1.185818
##   Coefficient of AR2: -0.4790522
```

```
plot_tempssm_residual_diagnostics(res_ar2)
```


Residuals



```
diag_lag12_ar2 <- diagnose_residuals(res_ar2, lb_lag = 12)
diag_lag24_ar2 <- diagnose_residuals(res_ar2, lb_lag = 24)
diag_lag36_ar2 <- diagnose_residuals(res_ar2, lb_lag = 36)
diag_lags_ar2 <- rbind(diag_lag12_ar2, diag_lag24_ar2, diag_lag36_ar2)
diag_lags_ar2$check <- c("lag 12", "lag 24", "lag 36")
diag_lags_ar2[, c("check", "lb_stat", "lb_lag", "lb_pvalue", "kurtosis")]
```

```
## # A tibble: 3 x 5
##   check lb_stat lb_lag lb_pvalue kurtosis
##   <chr>   <dbl> <dbl>   <dbl>   <dbl>
## 1 lag 12    9.93    12    0.623    3.66
## 2 lag 24   24.0    24    0.461    3.66
## 3 lag 36   43.9    36    0.170    3.66
```

なお、AR(2) モデルを感度確認として当てはめた場合、短いラグでの残差自己相関は限定的でしたが、残差 ACF では lag 24, 26, 27 付近に負の突出が認められました。この結果は、AR 次数を高くすることが、残差構造のすべての面を自動的に改善するわけではないことを示唆します。そのため、この例では AR(2) モデルを AR(1) ベースラインに対する明確な改善としてではなく、追加の診断的注意を要する候補モデルとしてみなせます。

これらの残差診断からは高次の AR 項が必要である強い根拠は認められないため、AR(1) モデルをベースラインの仕様として用います。

推定されたパラメータと潜在状態の成分

長期トレンド成分やその変化率（ドリフト）は、次のように `ts` オブジェクトとして 取り出すことができます。

```
# Smoothing estimates
alpha_hat <- res_ar1$kfs$alphahat
```

```
head(alpha_hat)
```

```
##           level      slope sea_dummy1 sea_dummy2 sea_dummy3 sea_dummy4
## Jan 1982 18.96708 0.002442910 -4.388514 -1.989138 0.8440593 3.3269013
## Feb 1982 18.96952 0.002442962 -5.783514 -4.388514 -1.9891376 0.8440593
## Mar 1982 18.97197 0.002442995 -5.905440 -5.783514 -4.3885143 -1.9891376
## Apr 1982 18.97441 0.002443203 -4.593786 -5.905440 -5.7835140 -4.3885143
## May 1982 18.97685 0.002443328 -1.903379 -4.593786 -5.9054404 -5.7835140
## Jun 1982 18.97930 0.002443456 1.457171 -1.903379 -4.5937863 -5.9054404
##           sea_dummy5 sea_dummy6 sea_dummy7 sea_dummy8 sea_dummy9 sea_dummy10
## Jan 1982 6.1507774 7.6637613 5.1211012 1.4571706 -1.903379 -4.593786
## Feb 1982 3.3269013 6.1507774 7.6637613 5.1211012 1.457171 -1.903379
## Mar 1982 0.8440593 3.3269013 6.1507774 7.6637613 5.121101 1.457171
## Apr 1982 -1.9891376 0.8440593 3.3269013 6.1507774 7.663761 5.121101
## May 1982 -4.3885143 -1.9891376 0.8440593 3.3269013 6.150777 7.663761
## Jun 1982 -5.7835140 -4.3885143 -1.9891376 0.8440593 3.326901 6.150777
##           sea_dummy11      arima1
## Jan 1982 -5.905440 0.27659369
## Feb 1982 -4.593786 0.17898925
## Mar 1982 -1.903379 0.48992423
## Apr 1982 1.457171 -0.08129112
## May 1982 5.121101 0.65071819
## Jun 1982 7.663761 1.09353211
```

```
# Smoothing estimate of level component
level_ts <- get_level_ts(res_ar1)

# Smoothing estimate of drift component
drift_ts <- get_drift_ts(res_ar1)

# Average drift rate per year across the full period
mean_drift_year <- mean(drift_ts)
print(mean_drift_year)
```

```
## [1] 0.03365529
```

SST の平均的な年上昇率は 0.0366 °C と推定されました。

短期予測

推定済みの tempssm オブジェクトは、`predict()` に渡すことで短期予測を取得できます。デフォルトでは、`predict(res)` は観測期間の終了直後の 1 時点先予測値を返します。この機能は、推定モデルが水準、季節成分、自己回帰成分をどのように外挿するかを可視化・点検する目的で有用です。

```
pred_1 <- predict(res_ar1)
pred_1
```

```
##           May
## 2026 18.71037
```

複数時点先の予測値を取得したい場合には、`n.ahead` 引数を指定します。

```
pred_12 <- predict(res_ar1, n.ahead = 12)
pred_12
```

```
##           Jan           Feb           Mar           Apr           May           Jun           Jul           Aug
## 2026                                18.71037 22.01798 25.65128 28.17714
```

```
## 2027 16.12028 14.72978 14.61284 15.92978
##           Sep      Oct      Nov      Dec
## 2026 26.65593 23.82915 21.34670 18.51594
## 2027
```

予測に伴う不確実性は、`interval` 引数を指定することで取得できます。`interval = "confidence"` は予測平均に対する信頼区間を返します。すなわち、予測される平均的な応答値に関する不確実性を表す下限値と上限値が返されます。一方で、`interval = "prediction"` は将来観測値に対する予測区間を返します。これは、将来の観測値そのものに関する不確実性を表す下限値と上限値であり、観測誤差も含まれるため、通常は `confidence interval` よりも `prediction interval` の方が広がります。信頼水準は `level` 引数で指定でき、デフォルトは 0.95 です。

```
pred_12_pi <- predict(
  res_ar1,
  n.ahead = 12,
  interval = "prediction",
  level = 0.95
)

pred_12_pi
```

```
##           fit      lwr      upr
## May 2026 18.71037 17.58819 19.83255
## Jun 2026 22.01798 20.68625 23.34972
## Jul 2026 25.65128 24.24035 27.06221
## Aug 2026 28.17714 26.73244 29.62184
## Sep 2026 26.65593 25.19542 28.11643
## Oct 2026 23.82915 22.36046 25.29783
## Nov 2026 21.34670 19.87329 22.82011
## Dec 2026 18.51594 17.03948 19.99240
## Jan 2027 16.12028 14.64169 17.59886
## Feb 2027 14.72978 13.24938 16.21017
## Mar 2027 14.61284 13.13087 16.09480
## Apr 2027 15.92978 14.44637 17.41319
```

これらの予測値は、確定的な将来予測ではなく、モデルに基づく外挿として解釈してください。一般に、予測期間が長くなるほど不確実性は大きくなり、長期の予測はトレンド、季節性、自己回帰構造に関するモデル仮定の影響を受けやすくなります。

演習 II: 外生変数を伴う温度時系列への状態空間モデルの適用

目的

状態空間モデリングのフレームを拡張して、外生的要因が温度変動に与える影響を検討します。具体的には、外生変数としての太平洋十年規模振動 (Pacific Decadal Oscillation; PDO) が、山口県沖で観測された SST に与える影響を検討します。

PDO 指数データセットの読み込み: 外生変数としての PDO 指数

- データ: 月別の太平洋十年規模振動 (PDO) 指数 (JMA)
- 期間: 1901 年 1 月から 2025 年 12 月

太平洋十年規模振動 (PDO) 指数は、20°N 以北の北太平洋における SST 変動の第 1 経験直交関数 (EOF) に、月平均 SST 偏差を射影したものとして定義されます。EOF は、1901 年から 2000 年の月別平年値を基準として定義された、1901–2000 年の SST 偏差を用いて計算されます。地球温暖化シグナルを取り除くため、EOF 解析に先立ち、各格子点から全球平均 SST 偏差が差し引かれます。本パッケージでは、気象庁 (JMA) が提供

する PDO 指数を使用します。データは https://www.data.jma.go.jp/kaiyou/data/shindan/b_1/pdo/pdo.txt で入手できます。

```
data(pdo) # load a ts object of PDO index
head(pdo)
```

```
##           Jan      Feb      Mar      Apr      May      Jun
## 1901  1.0040  0.7403  0.9011 -0.0109 -0.2325 -0.6810
```

温度時系列と PDO 時系列の共通期間への切り出し

外生変数を伴う状態空間モデリングでは、すべての入力時系列が共通かつ整合した時間インデックスをもつ必要があります。このステップでは、温度時系列と PDO 時系列について、先頭および末尾の重ならない部分を切り落とし、両データセットが同一の期間の範囲となるようにします。

`tempssm::trim_ts_overlap()` 関数を用いて 2 つの `ts` オブジェクトを共通の時間軸にそろえ、共通期間のみを含む多変量時系列を返します。

```
# Generate an object on a shared timeline
yamaguchi_sst_trim <- trim_ts_overlap(yamaguchi_sst,
                                     pdo,
                                     temp_name = "Temp",
                                     exo_name="PDO")$temperature
```

```
pdo_trim <- trim_ts_overlap(yamaguchi_sst,
                           pdo,
                           temp_name = "Temp",
                           exo_name="PDO")$exogenous
```

```
start(yamaguchi_sst_trim)
```

```
## [1] 1982    1
```

```
end(yamaguchi_sst_trim)
```

```
## [1] 2025   12
```

```
start(pdo_trim)
```

```
## [1] 1982    1
```

```
end(pdo_trim)
```

```
## [1] 2025   12
```

海面水温と PDO 指数の時系列プロット

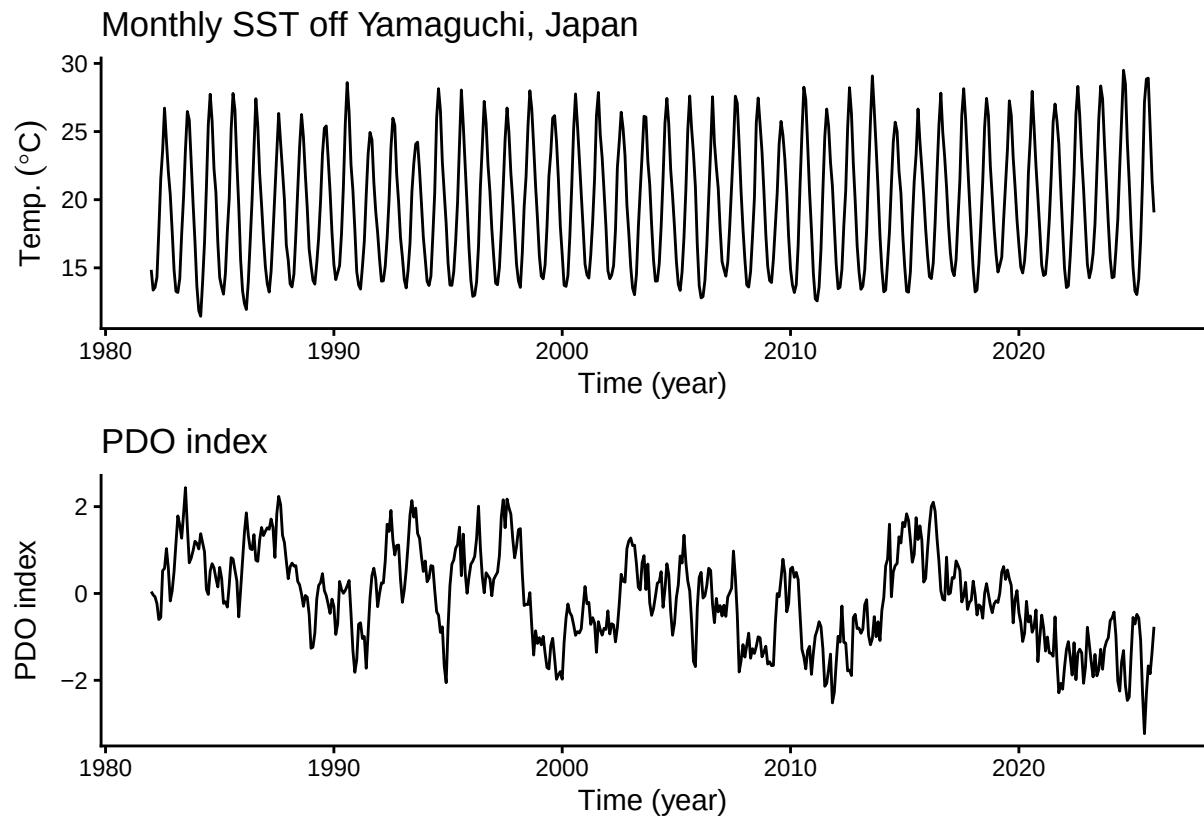
海面水温と PDO 指数の時系列構造を可視化して把握します。

```
plt_yamaguchi_sst_trim <- forecast::autoplot(yamaguchi_sst_trim) +
  labs(y = expression(Temp.~(degree*C)),
       x = "Time (year)") +
  ggtitle("Monthly SST off Yamaguchi, Japan") +
  theme_classic()
```

```
plt_pdo <- forecast::autoplot(pdo_trim) +
  labs(x = "Time (year)", y = "PDO index") +
```

```
ggtitle("PDO index") +
theme_classic()

plt_yamaguchi_sst_trim + plt_pdo + patchwork::plot_layout(ncol=1)
```



演

習用の PDO 指数データには欠損値はありません。ただし、ユーザー自身の外生変数データを用いる場合には、外生変数に欠損値が含まれていないことを事前に確認してください。tempssm() では、欠損値を含む外生変数は許容されず、そのようなデータを指定するとモデル推定はエラーで停止します。必要に応じて、解析前に適切な欠損値補完などの前処理をおこなってください。

一方で、従属変数である温度時系列データの欠損値は許容されます。状態空間モデルでは、これらの欠損値は未観測の応答値として扱われ、カルマンフィルタリングおよび平滑化の枠組みの中で推定が実行されます。

外生変数を含まないモデルの適用

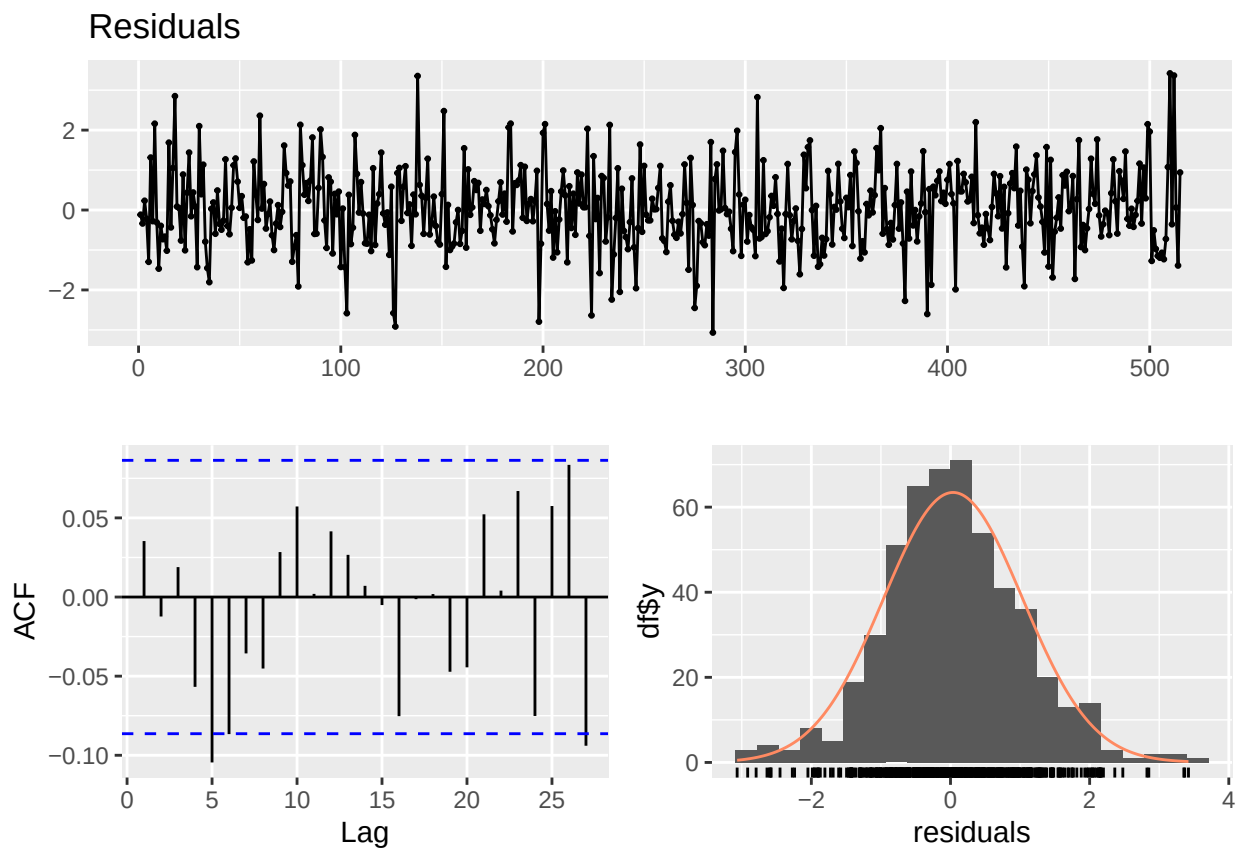
まず、外生変数を含まないベースラインの状態空間モデルを適用します。このモデルは、温度変動が潜在的な長期トレンド、季節周期、自己回帰的依存性のみで説明される参照ケースとして機能します。既に演習 I で同一のモデルを取得していますが、ここでは従属変数の引数を明示して実行し、戻り値を res_without とします。

```
res_without <- tempssm(temp_data = yamaguchi_sst_trim, ar_order = 1)
summary(res_without)
```

```
## tempssm summary
## -----
## Call:
## tempssm(temp_data = yamaguchi_sst_trim, ar_order = 1)
##
## Model fit:
## Likelihood type: marginal
## Log-likelihood : -435.46
```

```
## k : 5
## Diffuse states : 13
## Converged : TRUE
##
## Variance parameters:
## Observation (H): 2.019273e-14
## State (Q trend): 1.118445e-07
## State (Q season): 2.312771e-149
## State (Q ar): 0.3164652
##
## Components of auto-regression:
## Order of AR: 1
## Coefficient of AR1: 0.6205502
```

```
plot_tempsm_residual_diagnostics(res_without)
```



```
diag_res_without <- diagnose_residuals(res_without)
print(diag_res_without)
```

```
## # A tibble: 1 x 4
## lb_stat lb_lag lb_pvalue kurtosis
## <dbl> <dbl> <dbl> <dbl>
## 1 17.0 12 0.149 3.68
```

この AR(1) ベースラインモデルの残差診断では、lag 12 までの残差自己相関に有意性は認められません。この結果から、外生変数を含まないモデルにおいても、デフォルトの 1 次自己回帰構造は妥当な出発点であると考えられます。

このベースラインモデルは、次節で導入する PDO 指数の追加的な説明力を評価するための有用な基準となり

ます。

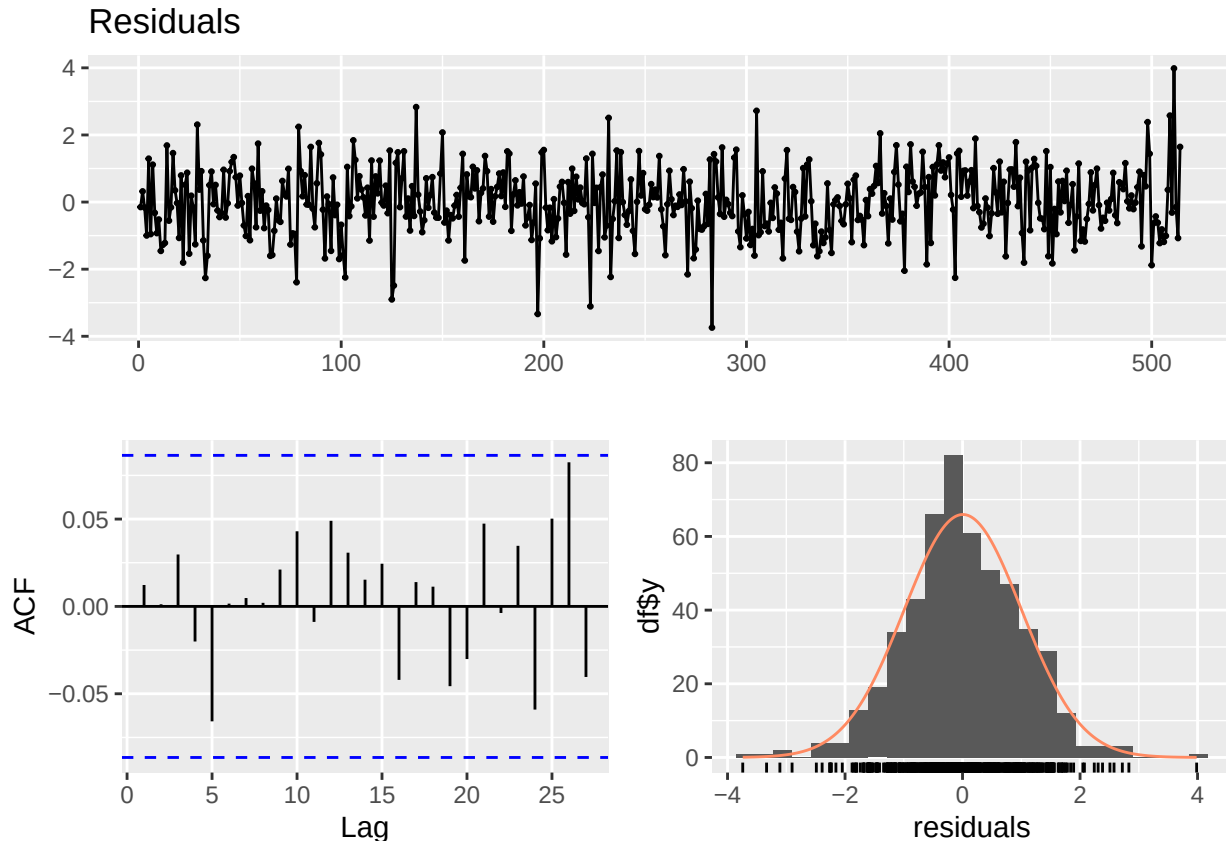
外生変数を含むモデルの適用

PDO 指数を単変量の外生変数とするモデルを構築します。tempssm() 関数において、外生変数用の引数 (exo_data) に PDO 指数のデータ (pdo_trim) を与えます。これまで明示していませんでしたが、温度時系列データ (yamaguchi_sst_trim) は 従属変数の引数 (temp_data) に 与えます。

```
res_with <- tempssm(temp_data = yamaguchi_sst_trim,
                    exo_data = pdo_trim,
                    ar_order = 1)
summary(res_with)

## tempssm summary
## -----
## Call:
## tempssm(temp_data = yamaguchi_sst_trim, exo_data = pdo_trim,
##       ar_order = 1)
##
## Model fit:
##   Likelihood type: marginal
##   Log-likelihood : -390.99
##   k               : 6
##   Diffuse states : 14
##   Converged      : TRUE
##
## Variance parameters:
##   Observation (H): 1.490122e-25
##   State (Q trend): 7.158619e-20
##   State (Q season): 2.316533e-82
##   State (Q ar): 0.2695508
##
## Components of auto-regression:
##   Order of AR: 1
##   Coefficient of AR1: 0.6566528
## Exogenous variable   PDO
## Estimated coefficient -0.4476073
## Lower CI  -0.5372882
## Upper CI  -0.3579264

plot_tempssm_residual_diagnostics(res_with)
```



```
diag_res_with <- diagnose_residuals(res_with)
print(diag_res_with)
```

```
## # A tibble: 1 x 4
##   lb_stat lb_lag lb_pvalue kurtosis
##   <dbl>   <dbl>   <dbl>   <dbl>
## 1     5.53     12     0.938     3.69
```

PDO 指数を含むモデルの残差診断でも、lag 12 までの残差自己相関に有意性は認められません。したがって、ここで比較する外生変数なし/ありの両モデルにおいて、AR(1) は残差診断上、作業モデルとして十分な自己回帰構造を与えていると考えられます。

モデルの推定結果を確認します。外生 PDO 指数の推定係数は負 (-0.45) であり、その 95% 信頼区間はゼロを含まず、-0.54 から -0.36 の範囲でした。これは、山口県沖の局所的な SST と PDO の変動との間に、統計的に有意な負の関係があることを示しています。

具体的な解釈として、基礎となる長期トレンド、季節周期、自己回帰的依存性を考慮した後も、PDO 指数が 1 単位増加すると、月別 SST は平均して約 0.45 °C 低下することがモデルから示唆されます。この結果は、PDO の正の位相が、調査地点における低温の条件に体系的に寄与することを意味します。

重要な点として、この外生変数の効果は、長期トレンドや時間依存性の表現が改善されたことによる見かけの効果としてではなく、温度時系列の内部動態に加えられた、もう一つ要因として識別されています。外生変数を含まないベースラインモデルと比べて PDO 係数が統計的に有意であることは、PDO が局所的な温度変動に影響する独立した大規模気候駆動因子として作用していることを示します。

外生変数を含むモデルで `predict()` を用いる場合には、将来時点の外生変数の値が必要です。利用可能な将来の外生変数値がある場合には、`new_exo_data` に指定できます。また、1 時点先の簡易的な可視化・点検であれば、`exo_strategy = "last"` により、外生変数の最終観測値をそのまま持ち越した予測を実行できます。これは外生変数に対する持続性の仮定であり、PDO 指数そのものの予測モデルではありません。


```
pred_with_last_exo <- predict(res_with, exo_strategy = "last")
pred_with_last_exo
```

```
##           Jan
## 2026 16.41488
```

時系列交差検証 (tsCV)

時系列交差検証 (tsCV) は、データの時間的な順序を保ちながら、標本外予測誤差に基づいてモデル性能を評価する方法です。そのため、モデルの妥当性について、追加的かつ独立した観点を提供します。

時系列交差検証では、拡張もしくはスライドする訓練期間に対してモデルを繰り返し当てはめ、その後続の観測値に対する予測性能を評価します。ランダムな交差検証とは異なり、未来から過去への情報漏洩が防がれるため、時系列データのモデル検証に適しています。

ここでは外生 PDO 変数を含むモデルと含まないモデルの交差検証指標を比較することで、PDO 係数に認められた統計的なシグナルが、標本外予測性能にも反映されているかを評価します。

```
# (Optional) Load packages for parallel processing.
# These are only required if you want to enable parallel execution:
# - future: controls how parallel workers are launched
# - future.apply: provides parallelized apply functions
# - progressr: optional progress bar support
library(future.apply)
library(future)
library(progressr)

if (interactive()) {
  future::plan(multisession)
} else {
  future::plan(sequential)
}

# ts cross-validation of the model without exogenous variables

## Generate a list of training and test datasets with their indices
# Procedure for constructing year-based time-series cross-validation folds:
#
# First training data: January 1982–December 2017;
# First test data: one year starting from January 2018
#
# Second training data: January 1983–December 2018;
# Second test data: one year starting from January 2019
# ...
#
# Eighth training data: January 1989–December 2024;
# Eighth test data: one year starting from January 2025
#
# These folds are automatically generated by the ts_train_test_split() function.

# Generate training and test dataset for model without PDO index
folds_without <- ts_train_test_split(
  temp_data = yamaguchi_sst_trim,
  exo_data = NULL,
```

```

initial = 432, # 432 monthly observations from Jan 1982 to Dec 2017
horizon = 12, # forecast 12 monthly time-series
step = 12, # training data is moved in one-year steps
fixed_window = TRUE,
allow_partial = FALSE
)

# Generate training and test dataset for model with PDO index
folds_with <- ts_train_test_split(
  temp_data = yamaguchi_sst_trim,
  exo_data = pdo_trim,
  initial = 432, # 432 monthly observations from Jan 1982 to Dec 2017
  horizon = 12, # forecast 12 monthly time-series
  step = 12, # training data is moved in one-year steps
  fixed_window = TRUE,
  allow_partial = FALSE
)

# *****
# Executing time-series cross validation

#-----
# Model without the exogenous variable of PDO index

# Single processing
# cv_without_results <- ts_cv_run(folds_without, ar_order = 1, use_season = TRUE)

# Parallel processing
cv_without_results <- future.apply::future_lapply(
  folds_without,
  ts_cv_run_fold,
  ar_order = 1,
  use_season = TRUE,
  future.seed = TRUE
)

# Computing assessment indexes
metrics_without <- lapply(cv_without_results, compute_cv_metrics)

# tidy summary
cv_without_tbl <- ts_cv_collect(cv_without_results, metrics_without) %>%
  mutate(Model="Without")

#-----
# Model with the exogenous variable of PDO index

# Single processing
# cv_with_results <- ts_cv_run(folds_with, ar_order = 1, use_season = TRUE)

# Parallel processing
cv_with_results <- future.apply::future_lapply(

```

```

folds_with,
ts_cv_run_fold,
ar_order = 1,
use_season = TRUE,
future.seed = TRUE
)

# Computing assessment indexes
metrics_with <- lapply(cv_with_results, compute_cv_metrics)

# tidy summary
cv_with_tbl <- ts_cv_collect(cv_with_results, metrics_with) %>%
  mutate(Model="With")

cv_tbl <- bind_rows(cv_without_tbl, cv_with_tbl)

cv_comparison <- compare_ts_cv(
  list(
    Without = cv_without_tbl,
    With = cv_with_tbl
  )
)

cv_comparison %>% knitr::kable()

```

model	n_folds	con- verged_n	converged_rate	mean_MAE	mean_MASE_naive	mean_MASE_seasonal
Without	8	8	1	0.6057671	0.2694420	0.7993162
With	8	8	1	0.5498521	0.2443207	0.7241733

```

plt_MAE <- ggplot(data=cv_tbl,
                  aes(x=Model,y=MAE)) +
  geom_boxplot()

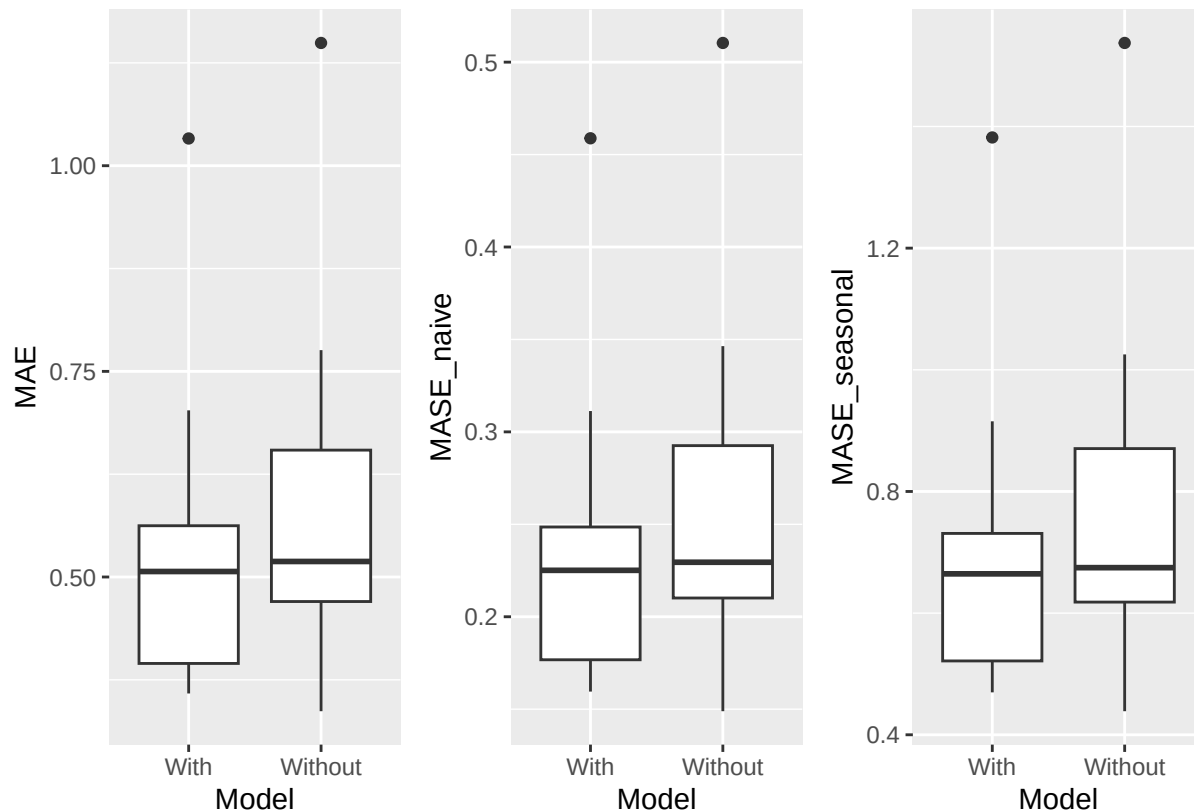
plt_MASE_naive <- ggplot(data=cv_tbl,
                        aes(x=Model,y=MASE_naive)) +
  geom_boxplot()

plt_MASE_seasonal <- ggplot(data=cv_tbl,
                           aes(x=Model,y=MASE_seasonal)) +
  geom_boxplot()

plt_tsCV <- plt_MAE + plt_MASE_naive + plt_MASE_seasonal + patchwork::plot_layout(nrow=1)

plot(plt_tsCV)

```



tsCV の解析結果は、外生変数を含むモデルが、外生変数を含まないモデルよりも良好であることを示しています。compare_ts_cv() によって作成された比較表では、各モデルの fold 数、収束率、および平均的な予測誤差指標を確認できます。一方で、箱ひげ図からは、検証期間ごとの誤差のばらつきを確認できます。このチュートリアルでは、実行時間を短縮するため、tsCV の反復回数を 8 回に設定しています。実際の検証では、十分な回数の反復を行ってください。

以上の結果を総合すると、状態空間モデルに外生変数として PDO 指数を含めることが一貫して支持されます。PDO 係数は統計的に有意であり、温度変動に対する明確な効果を示しています。さらに、時系列交差検証により、この効果が標本外予測性能の向上にもつながっていることが確認されます。

これらの相補的な証拠は、外生変数を含むモデルを採用するための頑健で多面的な根拠を提供します。

本演習では単一の外生変数を用いましたが、tempssm() は多変量の外生変数を含むモデルにも対応しています。関心のあるユーザーは、同様のワークフローを他の環境・気候・生態学的な共変量にも拡張できます。その際には、モデル診断や標本外予測性能を確認しながら、対象データに応じたモデル構造を慎重に検討してください。

外生変数あり/なしモデルの推定パターンの比較

長期トレンドとその変化率の推定パターンが、外生変数あり/なしモデルでどのように変わったかを見比べてみます。

```
plt_level_without_ts <- autoplot(res_without,
                                component=c("level")
                                ) +
  labs(title="Model without the PDO index") +
  theme_classic()

plt_drift_without_ts <- autoplot(res_without,
                                component=c("drift")
                                ) +
```

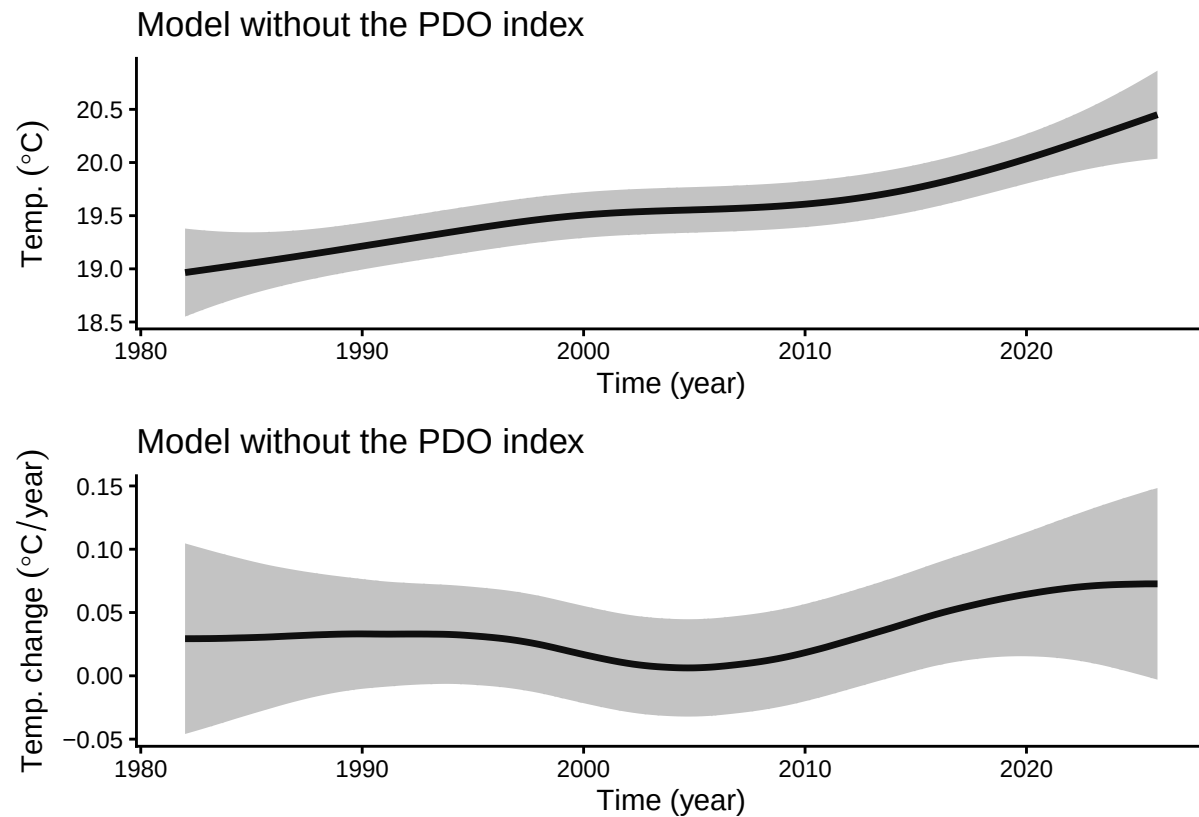
```

labs(title="Model without the PDO index") +
theme_classic()

plt_level_drift_without_ts <- plt_level_without_ts +
  plt_drift_without_ts +
  patchwork::plot_layout(ncol=1)

plot(plt_level_drift_without_ts)

```



```

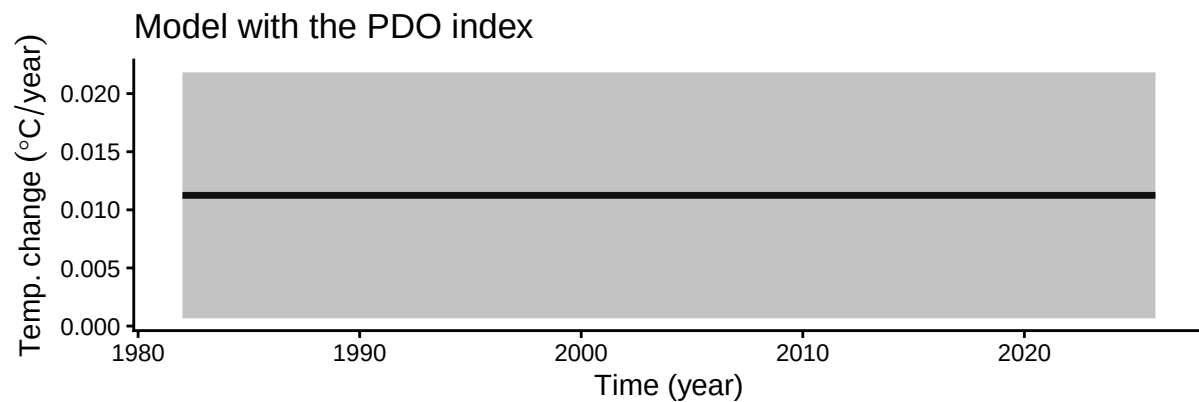
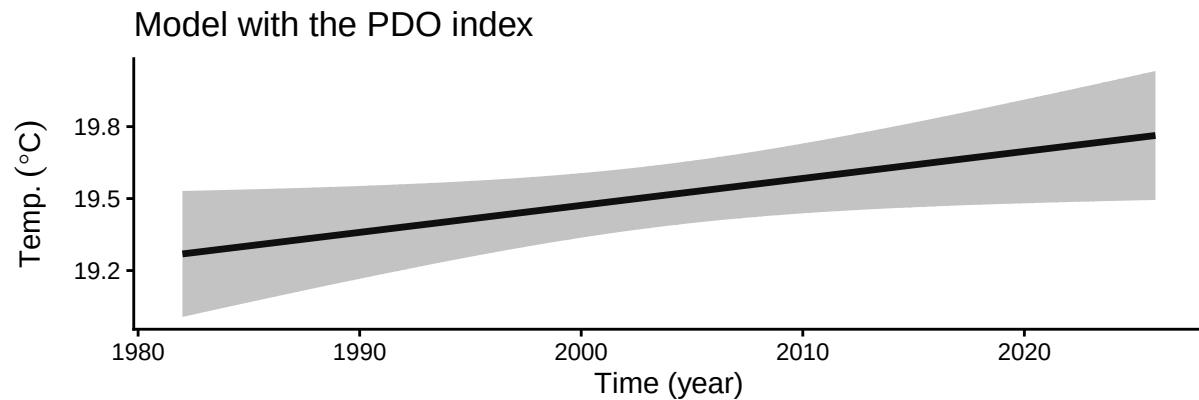
plt_level_with_ts <- autoplot(res_with,
                             component=c("level")
                             ) +
  labs(title="Model with the PDO index") +
  theme_classic()

plt_drift_with_ts <- autoplot(res_with,
                             component=c("drift")
                             ) +
  labs(title="Model with the PDO index") +
  theme_classic()

plt_level_drift_with_ts <- plt_level_with_ts +
  plt_drift_with_ts +
  patchwork::plot_layout(ncol=1)

plot(plt_level_drift_with_ts)

```



```
# Smoothing estimate of drift component
mean_drift_year_without <- get_drift_ts(res_without) %>%
  mean()
print(mean_drift_year_without)
```

```
## [1] 0.03388663
```

```
mean_drift_year_with <- get_drift_ts(res_with) %>%
  mean()
print(mean_drift_year_with)
```

```
## [1] 0.01124586
```

上の図の灰色の領域は、95% 信頼区間を示しています。

外生変数なしモデルでは、長期トレンドおよびその変化率のプロットに波打ったようなパターンが読み取れますが、外生変数ありモデルでは長期トレンドおよびその変化率はより安定的なパターンとなっています。つまり、この波打ったようなパターンは外生変数の PDO がおよぼす影響であることが示唆されます。

観測期間を通した SST の年あたりの変化率の推定値は、外生変数なしモデルでは 0.0339、外生変数ありモデルでは 0.0112 となりました。

今回のケースでは、SST のトレンドを正確に推定するには PDO 指数を組み込んだモデルの方が好ましいと考えられます。したがって外生変数なしモデルではやや大きめに推定されていた SST の上昇率が、外生変数ありモデルの導入によって、抑制的に推定されたことになります。

ユーザーのオリジナルデータの使用について

簡単な例

`my_ts` はユーザーが用意した温度時系列データの `ts` オブジェクトです。直接 `tempssm()` に渡すことができます。

```
## example of ts object (not run)
## my_ts <- ts(rnorm(100), frequency = 12)
res <- tempssm(my_ts)
```

ユーザーのオリジナルデータの準備

CSV 形式

月別温度時系列の CSV ファイルを、以下の列をもつ形式で準備します。- Year - Month - Temp

例:

```
Year,Month,Temp
2010,8,13.6
2010,9,6.8
2010,10,NA
2010,11,-1.4
...
```

- 欠測した温度値には NA を使用し、対応する Year と Month の行は必ず保持してください。
- CSV ファイルはカンマ区切りで、UTF-8 エンコーディングとしてください。

CSV から時系列への変換

本パッケージに同梱するサンプル CSV ファイルは `inst/extdata` にあります。このサンプルデータセットには、北海道赤岳 (1,840 m) における月別気温観測値が含まれています。オリジナルデータは環境省のモニタリングサイト 1000 から入手可能です。(KOZ01.zip、<https://www.biodic.go.jp/moni1000/findings/data/index.html>)。

```
path <- system.file("extdata", "example_monthly_temp.csv", package = "tempssm")
akadake_temp <- tempssm::read_monthly_temp_ts(path)
head(akadake_temp)
```

```
##           Jan Feb Mar Apr May Jun Jul   Aug   Sep   Oct   Nov   Dec
## 2010                                13.6   6.8   0.2  -6.8 -12.5
## 2011 -18.8
```

`read_monthly_temp_ts()` 関数は、内部で base R の `ts()` 関数を用いて時系列オブジェクトを作成します。より細かな制御が必要な場合は、手動で `ts` オブジェクトを作成することができます。

参考文献

`tempssm` に実装されている統計モデリングの枠組みは、Baba et al. (2024) に記述された方法論に基づいており、付随する補足資料およびコードリポジトリが本パッケージ開発の初期ソースとなっています。

Baba, S., Ishii, H., and Yoshiyama, T. (2024). Estimating sea temperature trends using a linear Gaussian state-space model in Jogashima, Kanagawa, Japan. *Bulletin of the Japanese Society of Fisheries Oceanography*, 88(3), 190–199. (日本語、英語要旨あり。) https://doi.org/10.34423/jsfo.88.3_190

Baba, S. (2024). 線形ガウス状態空間モデルを用いた海水温トレンド推定のための補足コードおよびテストデータ。GitHub リポジトリ: <https://github.com/logics-of-blue/sea-temperature-trend-jogashima>

Helske, J. (2017). KFAS: R における指数型分布族状態空間モデル。 *Journal of Statistical Software*, 78(10), 1–39. <https://doi.org/10.18637/jss.v078.i10>

付録: ユーティリティ関数

データ準備と探索的解析を支援するために、以下のユーティリティ関数が提供されています。

1. read_monthly_temp_ts()

`read_monthly_temp_ts()` 関数は、CSV ファイルに保存された月別温度データを R の `ts` オブジェクトへ変換します。単純で一貫したデータ形式を課すことで、外部で準備された時系列データを `tempssm` に取り込みやすくすることを目的としています。

例

月別温度データでは、ヘッダー行をもつ CSV ファイルを準備します。デフォルトでは、列名は `Year`、`Month`、`Temp` である必要があります。ここで `Temp` は観測された温度値を表します。

```
Year,Month,Temp
2001,1,10.4
2001,2,8.2
2001,3,NA
2001,4,13.6
2001,5,16.1
...
```

- 欠測した温度値には `NA` を使用し、対応する `Year` と `Month` の行は必ず保持してください。* CSV ファイルはカンマ区切りで、UTF-8 エンコーディングとしてください。

次の例では、パッケージに含まれるサンプル CSV ファイルを使用します。

```
tmp_csv <- tempfile(fileext = ".csv")
writeLines(
  c("Year,Month,Temp",
    "2001,1,10.4",
    "2001,2,8.2",
    "2001,3,NA",
    "2001,4,13.6",
    "2001,5,16.1"),
  tmp_csv
)

# Read the CSV file and convert to a monthly ts object
temp_ts <- read_monthly_temp_ts(tmp_csv)
```

2. convert_monthly_df_to_ts()

`convert_monthly_df_to_ts()` 関数は、月別温度データを含むデータフレームを R の `ts` オブジェクトへ変換します。この関数は、温度データをまずデータフレームとして読み込みまたは準備し、その後に時系列解析で使用するワークフローを支援するために設計されています。

入力データフレームには、少なくとも日付列 (`Date`) と温度列 (`Temp`) の 2 列が含まれていることが想定されています。なお日付列 (`Date`) について、月別の温度の集計値と対応させるために、日は任意の日数 (たとえば 1 日) を与えてください。

例

```
# Create a data frame of monthly temperature data
df <- data.frame(
  Date = as.Date(c(
    "2001-01-01",
    "2001-02-01",
    "2001-03-01",
    "2001-04-01",
    "2001-05-01")
  ),
  Temp = c(10.4, 8.2, NA, 13.6, 16.1)
)

# Convert to a monthly ts object
temp_ts <- convert_monthly_df_to_ts(df)
head(temp_ts)
```

```
##           Jan  Feb  Mar  Apr  May
## 2001 10.4   8.2   NA 13.6 16.1
```

3. get_jma_sst_ts()

get_jma_sst_ts() 関数は、気象庁（JMA）が提供する日本沿岸域の日平均海面水温（SST）データをダウンロードします。日別値を月平均へ集計し、得られた時系列を ts クラスのオブジェクトとして返します。

例

次の例では、茨城県南部沿岸域の SST データをダウンロードする方法を示します。

引数 sea_area_id は、JMA によって定義された海域の数値 ID を指定します。例えば、138 は茨城県南部沖の沿岸域に対応します。利用可能な海域 ID と対応する地域の一覧は、JMA により以下で提供されています。

https://www.data.jma.go.jp/kaiyou/data/db/kaikyo/series/engan/eg_areano.html

```
sst_138_ts <- get_jma_sst_ts(sea_area_id = 138)
head(sst_138_ts)
```

```
##           Jan      Feb      Mar      Apr      May      Jun
## 1982 15.04419 14.22500 13.63903 15.31933 17.52258 19.52300
```

4. compute_temp_anomaly()

compute_temp_anomaly() 関数は、R の ts オブジェクトとして与えられた時系列から偏差を計算します。偏差は、各観測値から対応する周期単位（たとえば月）の長期平均を差し引くことで計算され、周期の影響を取り除いた年変動パターンの確認などに用いられます。

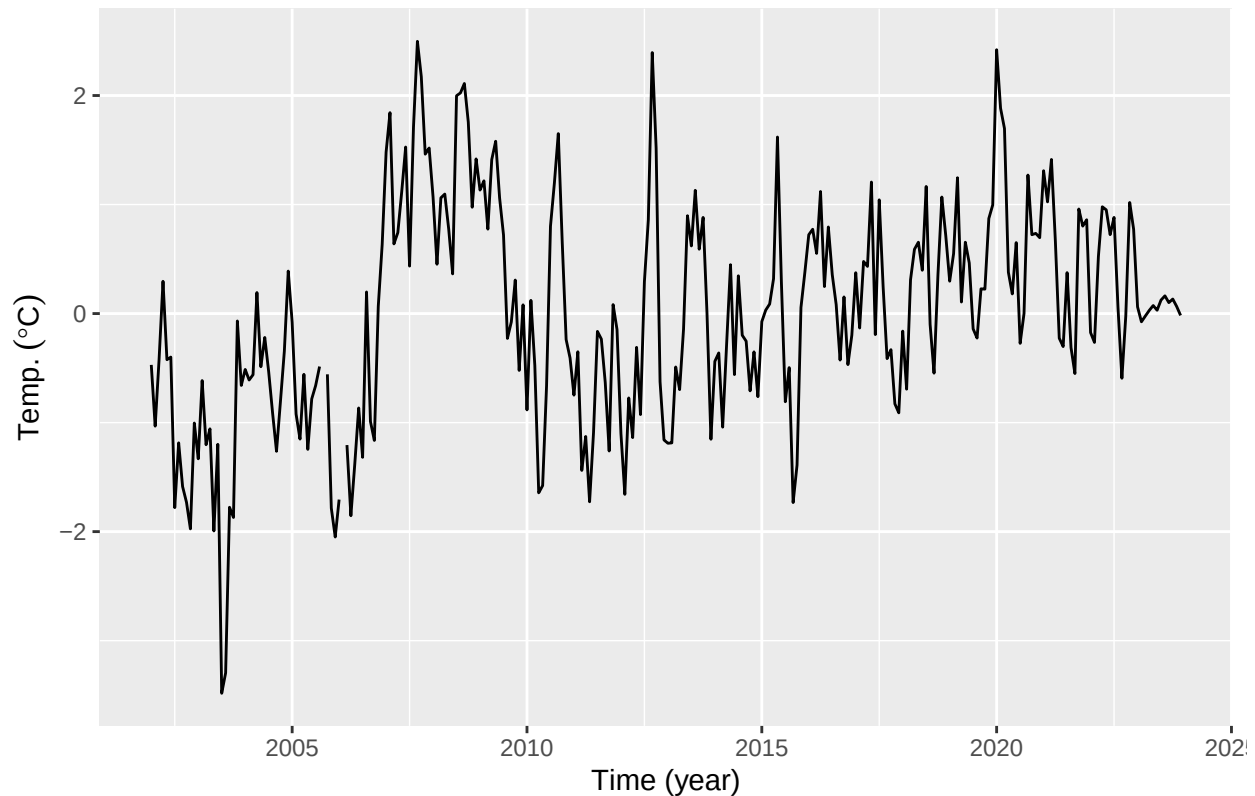
偏差を計算するための基準期間は、関数引数で指定できます。デフォルトでは、利用可能な時系列全体を用いて平年値が計算されます（?compute_temp_anomaly を参照）。

この変換は、典型的な季節条件からのずれに注目する探索的解析やモデリング用途で有用です。

例

```
# Generate temperature anomalies
data(niigata_sst)
niigata_sst_anomaly <- compute_temp_anomaly(niigata_sst)
```

```
plt_niigata_sst_anomaly <- forecast::autoplot(niigata_sst_anomaly) +
  labs(y = expression(Temp.~(degree*C)),
       x = "Time (year)")
plot(plt_niigata_sst_anomaly)
```



5. compute_monthly_climatology()

`compute_monthly_climatology()` 関数は、`ts` オブジェクトから、各季節値を年をまたいで平均することにより、気候学的な平均季節周期を計算します。主として、温度時系列における季節構造の探索的解析と可視化を目的としています。

例

```
data(niigata_sst)
monthly_seasonal_cycle_niigata_sst <- compute_monthly_climatology(niigata_sst)
summary(monthly_seasonal_cycle_niigata_sst)
```

```
##      Month      Temperature
##  Min.   : 1.00   Min.   : 9.365
## 1st Qu.: 3.75   1st Qu.:11.169
## Median : 6.50   Median :16.318
## Mean   : 6.50   Mean   :17.036
## 3rd Qu.: 9.25   3rd Qu.:22.294
## Max.   :12.00   Max.   :26.873
```

```
plt_monthly_seasonal_cycle_niigata_sst <- ggplot(
  data = monthly_seasonal_cycle_niigata_sst,
  aes(x = Month, y = Temperature)
```

```

) +
  geom_point(size = 2) +
  geom_line(linetype = "dashed") +
  labs(
    title = "Monthly seasonal cycle of temperature",
    x = "Month",
    y = expression(Temp.~(degree*C))
  ) +
  scale_x_continuous(
    breaks = 1:12,
    labels = sprintf("%02d", 1:12)
  ) +
  theme_classic()

plot(plt_monthly_seasonal_cycle_niigata_sst)

```

